

PROJECT UNDERSTANDING & BUILD PLAN

RoomMates

"Split expenses. Stay organized. Live together better."

A shared-expense management platform for people living together — room codes, flexible splits, an auditable expense history, and settlement suggestions that minimise the number of payments.

Stack	MERN — React + Vite + Tailwind / Express + MongoDB
Status at writing	Greenfield — no code exists yet
Prepared for	Alok
Date	2 September 2026

1 What I Understand This Project To Be

RoomMates is **not** a roommate-finding or social app. It is a small, correctness-critical **financial ledger** wrapped in a friendly SaaS-style interface. Everything else — the dashboard, the charts, the notifications — is presentation on top of one core question:

THE ONE QUESTION THE APP ANSWERS

"Given everything this group has spent, who owes whom, and how few payments can settle it?"

The product model in one paragraph: a user registers, creates a **room** (a shared-expense group) and gets a unique **room code** such as `RM-7X92AB` . They share the code; others join with it. Anyone in the room logs expenses — who paid, how much, for what, and *which members it should be split between*. The system continuously maintains each member's net balance, shows how each person compares to the group, and proposes a minimal set of payments to settle up. Every expense is permanent: edits are recorded as revisions rather than overwrites, and deletions are soft. One user can belong to many rooms (Home, Office, Trip, Veg group, Non-veg group), and room data never mixes.

What makes this harder than it looks

A naive version of this app is a weekend build. The requirements you have written are not the naive version — three of them are genuinely hard, and they are the ones that decide whether the architecture holds up:

REQUIREMENT	WHY IT IS HARD
Historical accuracy when membership changes Spec §3, §29	If shares are computed at read time from <i>current</i> membership, then the moment a 5th person joins, every past expense silently re-splits 5 ways and last month's report changes. Old settled balances become wrong.
Edits must never destroy the previous value Spec §8, §9	Calculations must use the latest value while the audit trail keeps every prior one — so the expense document and its history cannot be the same record.
Flexible splits (equal / custom / percentage / subset) Spec §6, §7	Once a dinner can be split between 3 of 5 people, "total ÷ members" stops being anyone's fair share. The whole calculation model has to change shape. See §2.2 — this is the one correction I'd make to the spec.

Sections 2 and 3 of this document are my proposed answers to those three. The rest is the build plan.

2 The Three Core Design Decisions

2.1 Snapshot the split at write time — never recompute it at read time

This single decision solves the historical-accuracy requirement (§3, §29) completely, and it costs almost nothing.

When an expense is created, the server resolves the split *immediately* and stores the result as an immutable list of per-member shares on the expense document itself:

```
// One expense document, after the server resolves the split
{
  amount: 2000, paidBy: [{ user: "rohit", amount: 2000 }],
  splitType: "equal",
  shares: [                                // ← frozen at creation. Never recomputed.
    { user: "alok", amount: 500 },
    { user: "rahul", amount: 500 },
    { user: "aman", amount: 500 },
    { user: "rohit", amount: 500 }
  ]
}
```

Now a member joining or leaving is just a change to the *room*'s member list. It cannot touch a single past expense, because no past expense refers to "current members" — each one carries its own participant list. Historical reports are stable by construction, not by careful coding.

CONSEQUENCE

Every figure in the app — balances, totals, per-member spend, settlements, any date-ranged report — becomes a simple sum over these frozen share rows. There is no "recalculate history" code path to get wrong, because there is no recalculation.

We still store `joinedAt` / `leftAt` on each membership as the spec asks, but they serve reporting and display ("who was here in July?"), not the money math. Members who leave are marked inactive, never removed, so their name still renders on old expenses.

2.2 Balance = paid – owed. "Average" is a display metric, not the fair share.

ONE CORRECTION TO THE SPEC

§5 and §11 define fairness as *difference from the average* (`total ÷ members`). That is correct **only** when every expense is split equally among *all* members. But §6 and §7 explicitly require custom splits, percentage splits, and subset splits — the veg/non-veg dinner case. The moment one dinner is split between 3 of 5 people, the average is no longer what anyone actually owes, and a dashboard built on it will show numbers that don't reconcile with the settlement page.

The model that stays correct in every case:

```

paidi      = Σ amounts member i actually paid out
owedi      = Σ member i's share rows across all active expenses
settledi   = Σ confirmed settlements i paid - Σ i received

balancei = paidi - owedi + settledi

balance > 0 → member is owed money (creditor)
balance < 0 → member owes money (debtor)
Σ balance   = 0 ← invariant, assert this in tests

```

Note the third term: settlements adjust balances *without touching expense records*, exactly as §12 requires. Keep the average — it is a genuinely useful "am I spending more than the group?" signal, and your §11 comparison table is a good screen. Just label it as a comparison metric and drive "you owe / you are owed" from `balance`. When all splits happen to be equal, the two agree anyway.

2.3 Expenses are append-only: revisions, not overwrites

Three rules give you the full audit trail in §8 and §9 with very little machinery:

- **Edit** → write the old field values into an `ExpenseRevision` document, then update the expense and set `isEdited: true`. The expense holds the current truth; revisions hold the past.
- **Delete** → set `isDeleted: true` and `deletedAt`. Every query filters on `isDeleted: false`. Nothing is ever removed from the database.
- **Settle** → insert a `Settlement` document. Expenses are never modified by a settlement.

Because an edit changes the amount, it also changes the shares — so the edit path re-resolves the split and writes a new frozen `shares` array, with the previous one preserved in the revision. Authorization is strictly enforced server-side: only `expense.createdBy` may edit; creator or room admin may delete.

3 The Calculation Engine

Per §28, this lives in `server/src/services/calculation/` as pure functions that take plain data and return plain data — no database access, no Express, no React. That is what makes it unit-testable, and financial logic must be tested.

3.1 Settlement: greedy minimum cash flow

Given the balances, we want the fewest payments that zero everyone out. The standard approach:

1. Split members into **creditors** (balance > 0) and **debtors** (balance < 0).
2. Sort both by magnitude, largest first.
3. Match the largest debtor against the largest creditor. Transfer `min(|debt|, credit)`.
4. Whichever side hits zero drops out; repeat until all balances are zero.

This yields at most $n-1$ transactions for n members, and in practice far fewer. Using your own worked example from §5:

MEMBER	PAID	OWED (EQUAL SPLIT)	BALANCE	ROLE
Alok	₹400	₹500	-₹100	debtor
Rahul	₹200	₹500	-₹300	debtor
Aman	₹600	₹500	+₹100	creditor
Rohit	₹800	₹500	+₹300	creditor

The algorithm pairs Rahul (-300) with Rohit (+300), then Alok (-100) with Aman (+100):

RESULT — 2 TRANSACTIONS

Rahul → Rohit ₹300 • Alok → Aman ₹100

Your spec §5 sketches a 3-transaction settlement (Rahul→Aman ₹200, Rahul→Rohit ₹100, Alok→Rohit ₹200). Both are valid — every balance reaches zero — but the greedy version needs one payment fewer, which is what §5's own "minimize unnecessary transactions" asks for. Worth knowing the spec's illustration and the implementation will differ here.

3.2 Money is stored in paise, never in rupees

NON-NEGOTIABLE

Store all amounts as **integer paise** (₹500.50 → `50050`). JavaScript floats cannot represent 0.1 exactly; a percentage split of ₹1000 three ways in floating point leaves stray fractions that accumulate until balances no longer sum to zero and the settlement page shows ₹0.01 phantom debts. Convert to rupees only at the display layer.

Related: an equal split of ₹1000 among 3 gives 33333 / 33333 / 33333 paise = 99999, one paise short. The engine assigns remainder paise deterministically (first n participants get +1) so shares always sum *exactly* to the expense total. This is a required unit test.

3.3 Split resolution

SPLIT TYPE	RULE	VALIDATION (SERVER-SIDE)
Equal	Total ÷ number of selected participants, remainder distributed deterministically	At least 1 participant
Custom	Explicit amount per participant	Σ shares must equal expense total exactly
Percentage	Total × pct, remainder distributed deterministically	Σ percentages = 100; Σ resulting shares = total
Subset	Any of the above, over a chosen subset of members	Every participant is an active room member

The veg/non-veg scenario in §7 is just **subset split with tags** — no special-casing. A room has free-form member tags ("veg", "non-veg", "1st floor"); selecting a tag pre-selects those members in the Add Expense form. Users who want fully separate books can still create a second room, since the spec allows both.

4 System Architecture

React client — Vite + Tailwind

Pages · reusable components · React Router with protected routes · Context for auth + active room · lazy-loaded routes ·
Recharts for the three dashboard charts. No business logic here.

↑ REST over HTTPS, JWT in Authorization header (+ Socket.IO later)

Express API

routes → middleware (auth, room membership, role) → validators (Zod) → controllers. Controllers stay thin:
parse, delegate, respond.

↓

Service layer — including the calculation engine

All business rules live here. The calculation engine is a set of **pure functions** with no DB or HTTP dependency, so it can be tested directly. This is the heart of the app.

↓

Mongoose models → MongoDB Atlas

Users · Rooms (with embedded members) · Expenses (with frozen shares) · ExpenseRevisions · Settlements · Notifications.
Compound indexes on every hot query path.

THE RULE THAT KEEPS THIS MAINTAINABLE

Business logic never leaks upward. A controller that adds two numbers together is a bug in the making — if the number matters, a service computes it. This is what \$41 is really asking for, and it is also what lets the entire money model be tested without spinning up a server.

4.1 Repository layout

```
RoomMates/
├─ client/
│  └─ src/
│     ├─ components/      ui/ · expense/ · room/ · charts/
│     ├─ pages/           one folder per route
│     ├─ layouts/         PublicLayout · AppLayout (nav + room switcher)
│     ├─ context/         AuthContext · RoomContext (active room)
│     ├─ hooks/           useAuth · useRoom · useExpenses · useDebounce
│     ├─ services/        axios instance + one module per API resource
│     └─ utils/           money formatting, dates, validators
├─ server/
│  └─ src/
│     ├─ config/          db, env, cors
│     ├─ models/          6 Mongoose schemas
│     ├─ routes/ controllers/ middleware/ validators/
│     └─ services/
│        ├─ calculation/  ← pure, heavily tested
│        ├─ expense.service.js
│        └─ room.service.js
```

```

├── settlement.service.js
├── utils/      ApiError, asyncHandler, roomCode generator
├── docs/      this document + the spec
└── README.md

```

4.2 Data model at a glance

COLLECTION	KEY FIELDS	INDEXES
User	name, email, passwordHash, avatar, lastLogin	<code>email</code> unique
Room	name, code, owner, <i>members[]</i> {user, role, tags[], joinedAt, leftAt, isActive}, isArchived	<code>code</code> unique; <code>members.user</code>
Expense	roomId, createdBy, amount, description, category, date, paidBy[], shares[] , splitType, isEdited, isDeleted, receiptUrl	{ <code>roomId</code> , <code>isDeleted</code> , <code>date:-1</code> } ; text index on description
ExpenseRevision	expenseId, editedBy, previousData, newData, changedFields[], createdAt	{ <code>expenseId</code> , <code>createdAt:-1</code> }
Settlement	roomId, payer, receiver, amount, status (pending/paid/confirmed), paidAt, confirmedAt	{ <code>roomId</code> , <code>status</code> }
Notification	userId, roomId, type, message, read, createdAt	{ <code>userId</code> , <code>read</code> , <code>createdAt:-1</code> }

Members are embedded in Room (bounded, small, always read together). Expenses are their own collection (unbounded, paginated, filtered). Revisions are separate so the expense document stays small no matter how many times it is edited.

4.3 Authorization matrix

ACTION	MEMBER	ROOM ADMIN	RULE ENFORCED IN
Add expense to room	✓	✓	<code>requireRoomMember</code>
Edit an expense	own only	own only	<code>expense.service</code> — even admins cannot edit others' expenses (§8)
Delete (soft) an expense	own only	✓ any	<code>expense.service</code>
Remove a member, rename room, regenerate code, archive	X	✓	<code>requireRoomAdmin</code>
Mark settlement paid	payer only	payer only	<code>settlement.service</code>
Confirm settlement received	receiver only	receiver only	<code>settlement.service</code>

PER §31

Every one of these is checked on the server from the JWT subject. Room IDs, room codes, member IDs and `createdBy` values arriving from the client are treated as untrusted input, always.

5 How We Complete It — 13 Phases

Each phase ends with something that runs and can be checked. Estimates assume solo part-time work; halve them if you are on this full-time. **Total ≈ 7–9 weeks part-time.**

PHASE 1 Foundation & architecture

2–3 days

- Scaffold `client/` (Vite + React + Tailwind + Router) and `server/` (Express).
- MongoDB Atlas cluster, connection handling with retry, `.env.example`, config module.
- Cross-cutting pieces up front: `ApiError`, `asyncHandler`, global error middleware, consistent response envelope, CORS, the axios instance with interceptors.
- Git repo, `.gitignore`, README skeleton.

DONE WHEN — both apps start with one command each, client can hit `GET /api/health`, and a thrown error anywhere returns the standard JSON error shape.

PHASE 2 Authentication

3–4 days

- User model, bcrypt hashing, register / login / me endpoints, JWT issue + verify middleware.
- Rate limiting on auth routes; Helmet; input validation.
- Client: Landing, Login, Register, AuthContext, `<ProtectedRoute>`, 401 interceptor → redirect.

DONE WHEN — you can register, log out, log back in, and a protected page bounces you to login when the token is missing or expired.

PHASE 3 Rooms, codes & membership

4–5 days

- Room model with embedded members; collision-safe room-code generator (`RM-XXXXXX`).
- Create / join by code / list my rooms / room details / rename / regenerate code / remove member / archive.
- `requireRoomMember` and `requireRoomAdmin` middleware.
- Client: My Rooms, Create Room, Join Room, RoomContext + the room switcher in the nav.

DONE WHEN — two accounts land in the same room via a code, a user belongs to several rooms, and switching rooms swaps all on-screen data with no bleed between them.

PHASE 4 Expenses & split resolution

5–6 days

- Expense model with the frozen `shares[]` array (§2.1); paise-integer money handling.
- Split resolver for all four types + full server-side validation (§3.3).
- Create / list (paginated, filtered) / detail endpoints.
- Client: Add Expense form with member selection and tag shortcuts, expense list, expense detail.

DONE WHEN — every split type produces shares that sum exactly to the total, and a subset split (the veg dinner case) works end to end.

PHASE 5 Calculation engine

3–4 days

- Pure functions: per-member paid, owed, balance; group total; average; difference-from-average.
- Greedy minimum-cash-flow settlement generator (§3.1).
- `GET /api/rooms/:id/summary` returning the whole computed picture in one call.
- **Write the unit tests in this phase, not in phase 12.** This is the code most likely to be wrong and least likely to announce it.

DONE WHEN — the \$5 worked example reproduces exactly, balances always sum to zero across randomised inputs, and paise remainders never go missing.

PHASE 6 Dashboard

4–5 days

- Summary cards: total, my spending, average per person, I owe, owed to me.
- Member comparison table with below/near/above-average indicators (§11).
- Three Recharts views: spend by member, by category, over time.
- Recent expenses list, skeleton loaders, empty states.

DONE WHEN — the dashboard reconciles with the settlement page to the paise, and a brand-new room shows a helpful empty state rather than zeros.

PHASE 7 Editing, revisions & soft delete

3–4 days

- ExpenseRevision model; edit path writes a revision, re-resolves shares, sets `isEdited`.
- Creator-only edit enforced server-side; soft delete with audit fields.
- Client: Edit Expense, "Edited" badge, revision history drawer showing previous → new, who, when.

DONE WHEN — editing ₹500 → ₹650 shows the badge, the history still shows ₹500, dashboards use ₹650, and another user's edit attempt is rejected by the API (not just hidden in the UI).

PHASE 8 Settlements

3–4 days

- Settlement model and endpoints; pending → paid → confirmed lifecycle with the right actor for each step.
- Settle Up page: you owe / you will receive / suggested settlements, plus settlement history.
- Confirmed settlements feed back into `balance` — and never modify expenses (§12).

DONE WHEN — recording every suggested settlement drives all balances to zero and the Settle Up page reads "Everyone is settled up 🦺".

PHASE 9 History, search & filtering

3 days

- History page: filter by date range, member, category, amount, edited, deleted; four sort orders.
- Server-side pagination throughout; debounced search on description.
- Supporting compound indexes verified with `.explain()`.

DONE WHEN — a room seeded with 1,000 expenses filters and pages without the browser ever holding more than one page.

PHASE 10 Notifications & optional real-time

2–3 days

- Notification model, emitted from the service layer on expense added/edited, member joined, settlement paid.
- Bell menu with unread count and mark-as-read.
- *Optional*: Socket.IO room channels so live viewers update without refresh. Emitting from services means this can be added later without restructuring anything (§30).

DONE WHEN — adding an expense notifies the other members; if sockets are in, a second browser updates without a refresh.

PHASE 11 Responsive polish & performance

4–5 days

- Real mobile layouts — tables become cards, bottom nav, thumb-reachable forms (§22).
- Splash screen (1–2s, skipped when already loaded), toasts, confirm dialogs, consistent empty states.
- Lazy-loaded routes, bundle audit, image handling; accessibility pass — labels, focus states, keyboard nav, contrast (§37).

DONE WHEN — every page is usable one-handed at 360px wide, and the whole app is navigable by keyboard alone.

PHASE 12 Security hardening & test completion

3–4 days

- Deliberate attempt to break your own authorization: edit someone else's expense by ID, read a room you never joined, post an expense naming a non-member. All must fail at the API.
- NoSQL-injection guards, CORS lockdown, rate limits, Helmet, sanitised error responses.
- Integration tests over the auth and authorization paths, completing the §35 checklist on top of the phase-5 unit tests.

DONE WHEN — every item in §35 has a passing test and no crafted request gets past an authorization check.

PHASE 13**Seed data, docs & deployment**

2–3 days

- Seed script: demo users, room, expenses, settlements — kept out of production code paths (§34).
- README with real setup commands; API reference; environment variable table.
- Deploy client to static hosting, server to a Node host, database on Atlas; production env vars and CORS.

DONE WHEN — a stranger can clone the repo and be running locally in under five minutes, and the deployed URL walks the full §43 acceptance workflow.

6 Risks & Open Decisions

6.1 Where this project is most likely to go wrong

RISK	MITIGATION
Floating-point money	Integer paise everywhere, enforced at the model layer. The single highest-value rule in this document.
Recomputing history	Frozen <code>shares[]</code> (§2.1). If you take one architectural idea from this plan, take this one.
Average vs. balance drift	Drive all money statements from <code>balance</code> ; treat the average as a comparison metric only (§2.2).
Client-side-only authorization	Hiding an Edit button is not security. Phase 12 exists specifically to attack this.
Scope creep	§39 is right — this is a ledger, not a social network. Phases 1–8 are the product; 9–13 make it shippable. Anything else is v2.
Building the UI before the engine	Phases are ordered deliberately: data model and money math before dashboards. §40 says the same thing.

6.2 Decisions worth making before phase 1

QUESTION	MY RECOMMENDATION
JavaScript or TypeScript?	TypeScript, if you are comfortable with it. In an app whose core is money arithmetic, having the compiler check that a share array actually carries <code>{user, amount}</code> pays for itself. If it would slow you down, plain JS with Zod validation at the boundaries is a fine second choice.
Refresh tokens?	Skip for v1. A single access token with a sane expiry and a clean 401 → login redirect meets §14. Add refresh later if session length becomes a real complaint.
Socket.IO in v1?	Optional, per §30. Emit notification events from the service layer from day one; wiring sockets in afterwards then costs about a day.
Receipt image uploads?	Cloudinary free tier. Never store binaries in MongoDB.
Multi-currency?	Not in v1. Fix on ₹ (INR). Retro-fitting currency into a ledger is painful, so if you think you will ever need it, add a <code>currency</code> field to Room now and leave it constant.

6.3 Suggested order of work

Phases 1 → 8 are strictly sequential; each genuinely depends on the one before. From phase 9 onward the order is flexible — if you have a demo or submission deadline, do phase 13 (seed data + README + deploy) early so there is always something running that you can show, then continue polishing.

THE SHORTEST PATH TO SOMETHING DEMONSTRABLE

Phases 1 → 5 plus a rough dashboard gives you the complete §43 acceptance workflow in about **2.5 weeks part-time**: register, create a room, join by code, add expenses, see balances and settlements. Everything after that raises quality rather than adding capability — which is the right shape for a project you may need to show before it is finished.